

基于TorchTitan的DeepSeek-V4昇腾续训练优化实践

作者：许虞俊 / 汪超 / 牛玉虎

时间：2026.4

DeepSeek-V4 模型亮点介绍

1M上下文

1.6T/284B 参数

>32T Tokens

1 MoE稀疏激活与路由优化

- 基于 MoE 架构实现稀疏激活
- 前 3 层采用 Hash Routing
- 开发 MoE 模块单一 MegaKernel，最大化 overlap 通信与计算

3 Hybrid Attention: CSA / HCA

- 融合滑动窗口、稀疏注意力、压缩注意力
- 提出 CSA / HCA 注意力机制
- 支持长序列 two-stage CP 并行
- 最大支持 1M 输入

5 Muon优化器

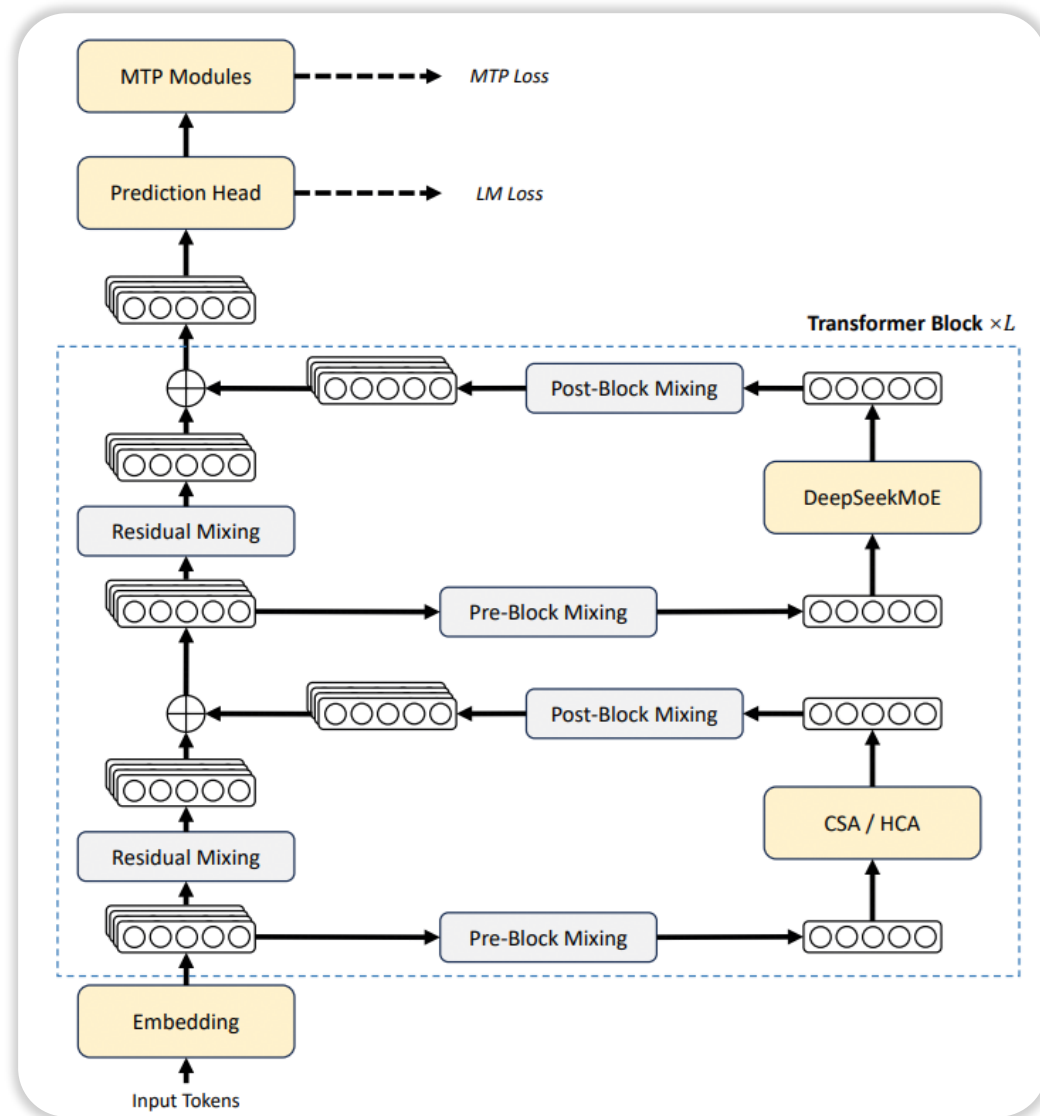
- 首次引入训练效率更高的 Muon 优化器
- 细粒度在不同模块应用 AdamW 与 Muon 双优化器

2 mHC增强残差连接

- 引入 mHC，在传统残差连接基础上进行扩展，提升训练稳定性
- 配套实现了 mHC fused kernels

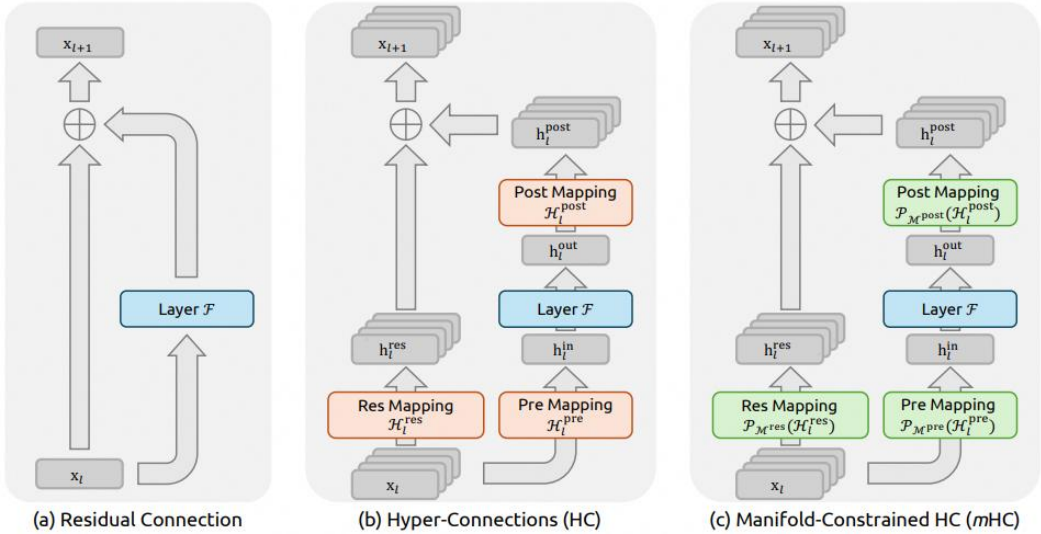
4 低精度训练与推理

- 预训练采用原生混合低精度 FP8 训练
- 在RL训练的rollout阶段，对 MoE 专家权重与 indexer 采用 FP4 低精度推理

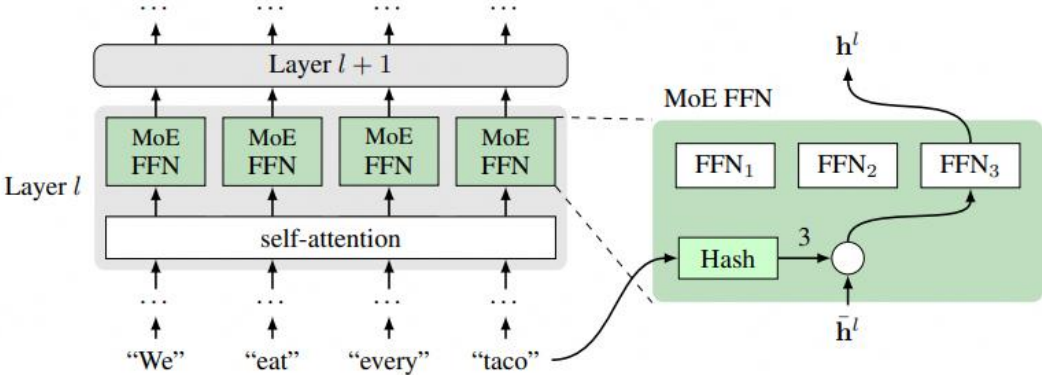


DeepSeek-V4 模型引入的新结构

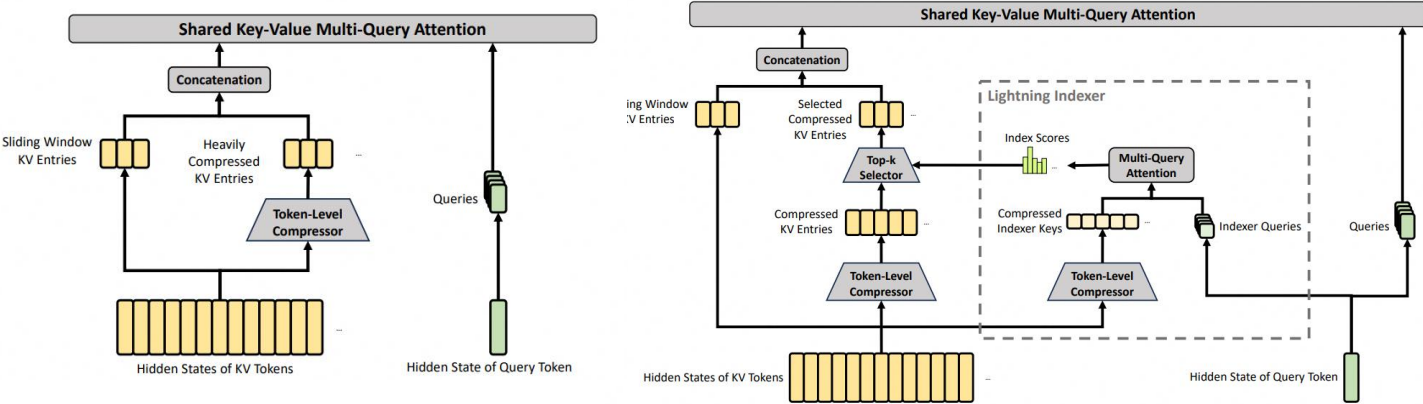
✓ 引入了mHC、全新的CSA和HCA结构以及hash routing，全面提升模型性能。



mHC通过引入双随机矩阵对多流混合过程施加流形约束，有效解决了传统超连接(HC) 由于没有残差连接恒等映射约束导致训练不稳定的问题。



为了解决浅层 MoE 路由不稳定的问题，在前 3 个 MoE 层中引入了 Hash Routing，根据 token ID 通过固定映射表直接分配 routed experts，让浅层 token 分配更加稳定和可控。



结合压缩注意力、滑动重叠窗口与稀疏注意力的高效注意力结构CSA和HCA。CSA 先按序列维压缩 KV cache，再进行稀疏注意力；HCA 采用更高压缩率压缩 KV cache，但保留 dense attention。高效解决大模型超长上下文场景下的注意力算力瓶颈问题。

DeepSeek-V4 模型带来的挑战



复杂结构的模型切分

- MLA/CSA/HCA等特殊结构会使得模型切分的适配变得复杂，需要较高维护成本



融合算子开发

- 面对快速迭代的模型，手写高性能融合算子的方式难以跟上节奏，开发周期长，无法及时响应业务需求



算子碎片化

- 复杂 MoE 与复杂数据流使计算图中细粒度算子增多，导致算子碎片化，极易触发 host-bound

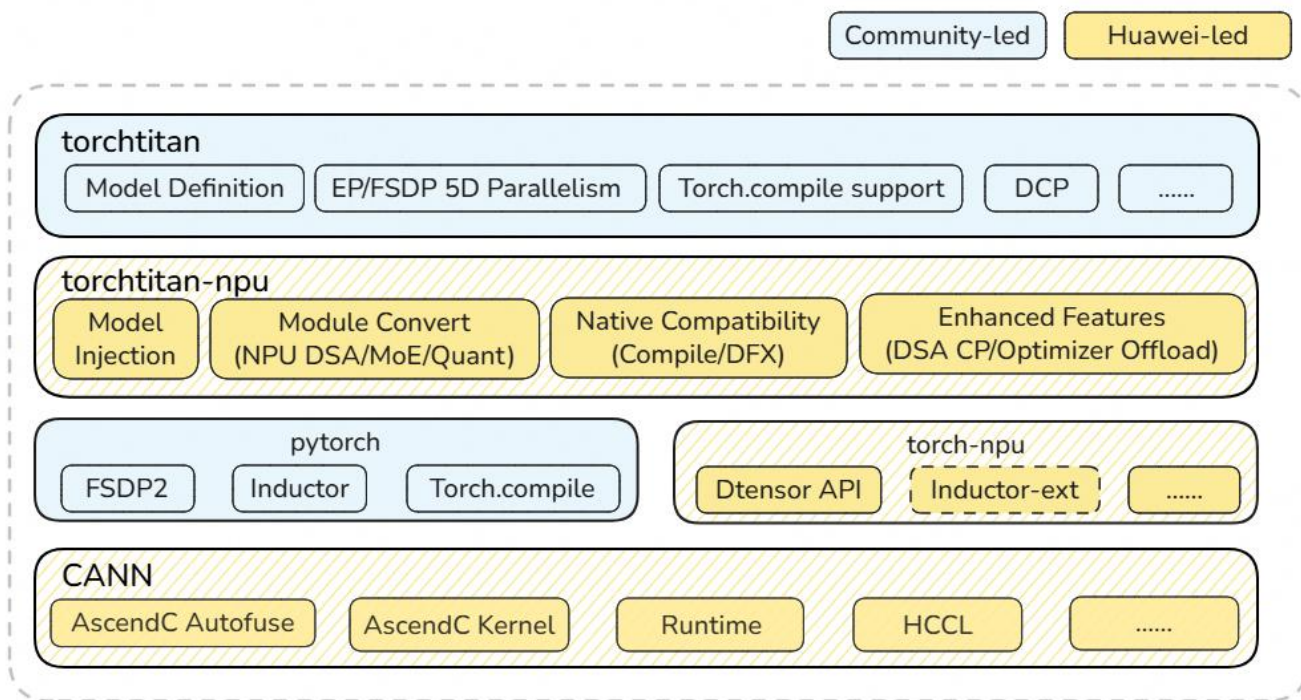


模型拓扑更复杂

- 模型拓扑越来越复杂（如 mHC），计算图呈现更深的数据依赖，Kernel 并行调度难度大大增加

DeepSeek-V4 续训练方案

- ✓ 依赖Atlas 800T A3超节点64卡，基于 torchtitan-npu 插件化适配 + AutoFuse 特性使能，使用**FSDP128** + **EP128** 分布式训练策略，在4K序列续训练场景最高吞吐达到 **1100 tokens/p/s**。

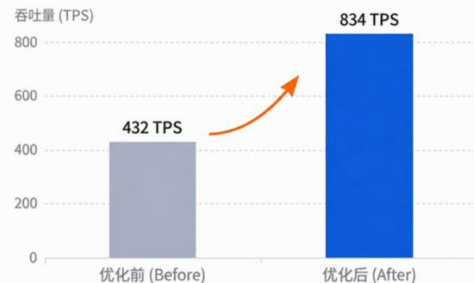


- 超节点亲和的纯FSDP+大EP切分
- 使能Sparse Flash MLA/LI等高性能NPU融合算子
- 使能torch.compile训练入图，叠加AutoFuse算子自动融合机制

TP	PP	VPP	EP	FSDP	MBS	GBS	集群
1	1	1	128	128	1	1024	64卡

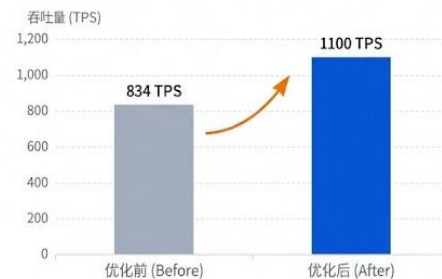
1. LI等高效NPU融合算子

整网吞吐提升效果：
↑ 90%
Speed up



2. 算子 AutoFuse 特性

整网吞吐提升效果：
↑ 30%
Speed up



为什么选择 torchtitan

torch原生分布式编程模型

- 将张量并行抽象为一种**第一公民语义**，**DTensor**支持 Shrad / Replicate / Partial 等 Placement 语义。
- **DTensor**基于完整 DeviceMesh，可以低侵入组合 **FSDP + TP + PP + EP + CP**，实现5D并行，保持代码紧凑、模块化。
- 模型切分与模型定义解耦，开发者主要声明子模块 ParallelStyle 来定义如何切分，**无需手写通信逻辑**。

PyTorch Native训练，畅享Torch"全家桶"

- torchtitan 是 PyTorch 社区原生大模型训练框架，便于**复用开源方案**并降低迁移门槛。
- 原生支持 **torch.compile + Inductor**，通过 FX 图变换、模式匹配和算子融合减少冗余开销。
- 将 PyTorch 多项原生能力(**DTensor、Compile、DCP、TorchAO、TorchFT**) 以系统化的方式有机整合，而非碎片化调用

TorchTitan-NPU插件化支持方案



免侵入式插件机制



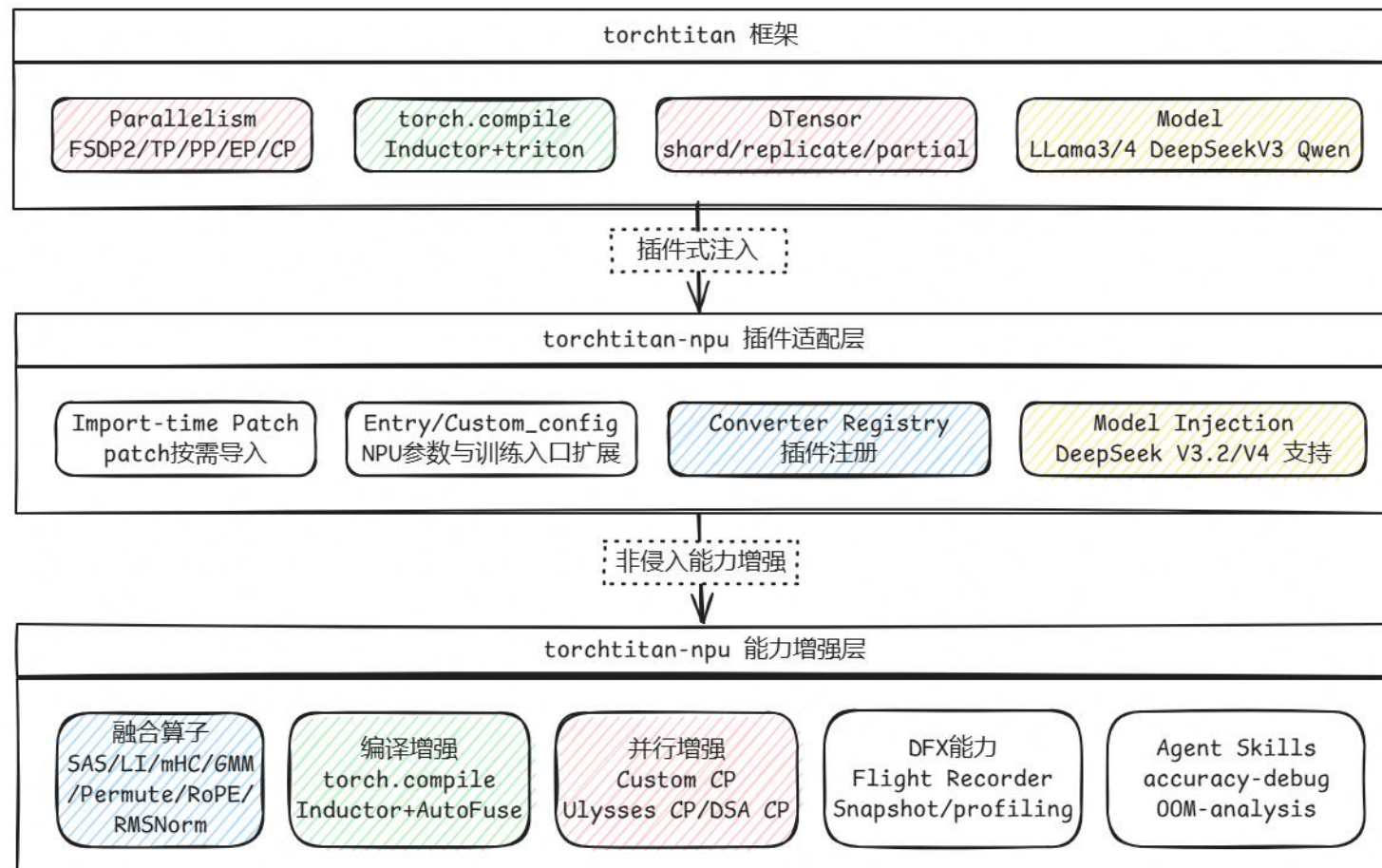
按需启用的开关配置



自由组合的优化矩阵



无缝对接 torch.compile+inductor



作为其昇腾后端扩展插件，**torch titan-npu** 基于 torch titan 官方 **ModelConverter** 机制构建，通过**即插即用式**使能 NPU 亲和性优化，复用 **inductor** 能力，叠加 AscendC 算子 codegen 后端 **AutoFuse**，实现与原生框架的能力对齐。

DeepSeek-V4 昇腾高性能续训练优化 Highlights



开箱即享的高性能续训练解决方案

CANN 已基于 TorchTitan 支持 DeepSeek-V4 的模型续训练，采用A3集群BF16精度64卡4K序列，MTP1训练吞吐达 **1100 Tokens/p/s**，MFU稳定在 **28.78%**



精简的并行切分策略

采用超节点亲和的**纯 FSDP + 大 EP** 的精简并行切分策略，以极低适配成本和通信开销达成内存占用最优，实现易用性与性能的较好均衡



高效的重型融合算子

针对稀疏注意力和mHC等复杂结构，定制 **Sparse Flash MLA**、**LightningIndexer** 等多个高效的 NPU 融合算子，充分释放芯片稀疏算力

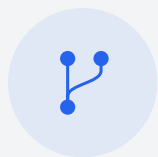


torch.compile + inductor + AutoFuse

依托Ascend C AutoFuse 特性实现端到端的 Vector 算子自动融合，为整网带来 **30%+** 的开箱即用性能收益

极简分布式并行：纯FSDP+大EP

传统模型切分面临的核心挑战



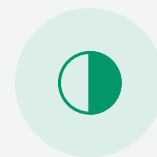
TP 切分逻辑复杂

MLA/CSA/HCA等特殊结构
使得张量并行切分更复杂
适配与维护成本高



PP 流水线难调优

Embedding、LmHead、MTP等模
块计算不均，易导致各 Stage 间耗
时失衡，流水线效率下降



MoE 动态负载不均

专家层的数据分布差异导致的**负载波动**，进一步放大PP的不均衡，
增加调优与收敛成本。



复杂模型下，传统混合并行容易出现TP切分复杂、PP bubble难消除、MoE负载不均等瓶颈。
为兼顾易用性与性能，我们进一步采用更简洁的“纯FSDP+大EP”并行方案

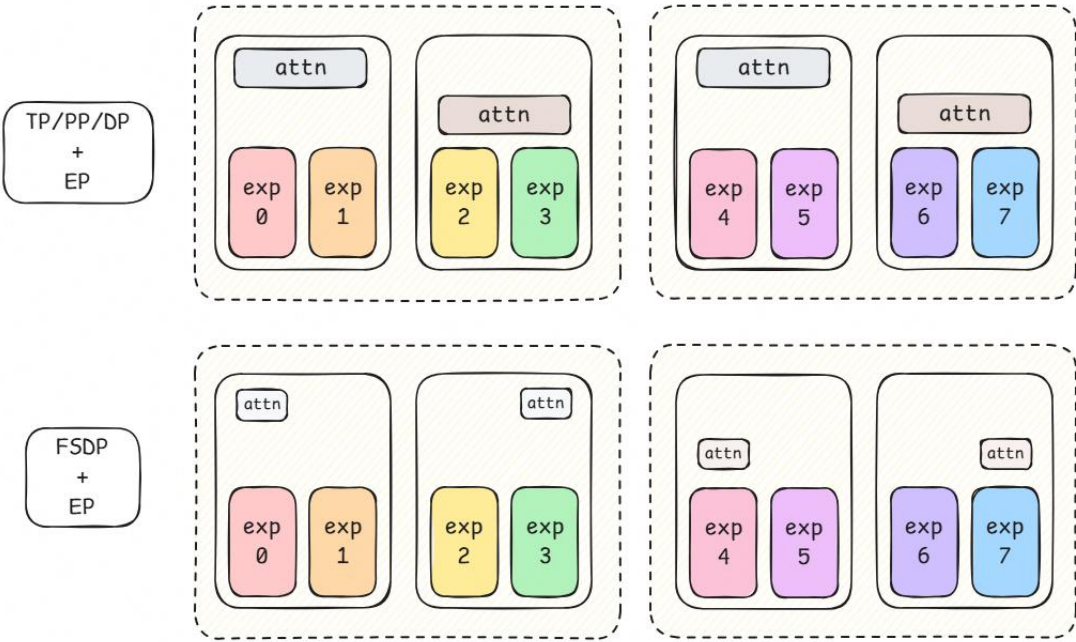
极简分布式并行：更少的未掩盖通信，更小的显存占用

我们的解决方案

摒弃复杂的TP/PP切分，全面采用**纯FSDP + 大EP**的极简并行方案，利用A3超节点**高通信带宽**，打破大模型部署瓶颈。

核心数据支撑

通信量分析：FSDP=128, EP=128配置下，未掩盖的通信仅占**12%**。
资源占用对比：专家显存占比从31.9%降至**2.8%**。



DeepSeek-V4 内存占用分析

DP	FSDP	TP	PP	EP	非专家 参数量	非专家 权重+梯度	专家参 数量	专家 权重+梯度	非专家内 存占比
16	1	2	4	128	1B	7.5GB	~2B	16GB	31.9%
1	128	1	1	128	0.06B	0.465GB	~2B	16GB	2.8%

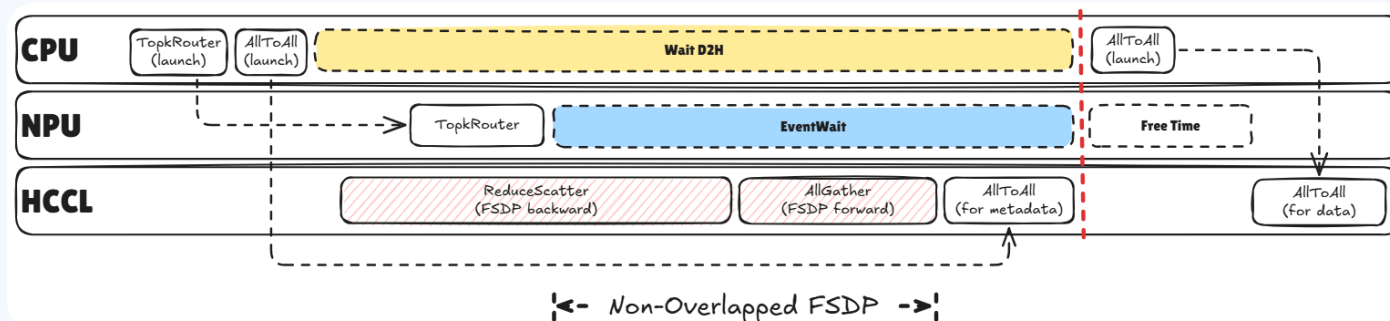
注：主权重和模型梯度均按照混合精度训练场景下常见的4字节大小计算

通算并发——通信开销被完全掩盖



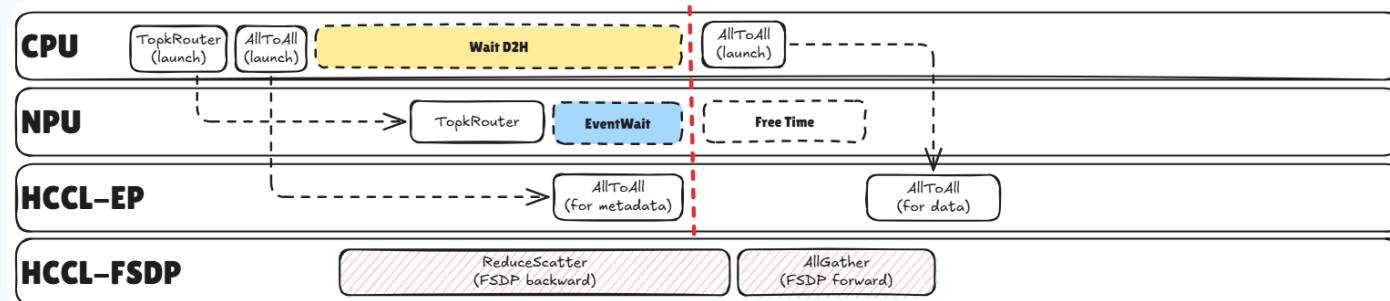
极简分布式并行：双通信域并行优化

FSDP128 + EP128场景下，FSDP域和EP域使用全局默认通信域，使得Pytorch为**FSDP**和**EP**分配同一个**ProcessGroup**，导致两者通信操作被迫串行执行，无法并发



通信并发

在MoE层的通信等待时长由最长10ms降低至~3ms，整网Speedup 1.1% ↑



DeepSeek-V4 昇腾高性能续训练优化 Highlights



开箱即享的高性能续训练解决方案

CANN 已基于 TorchTitan 支持 DeepSeek-V4 的模型续训练，采用A3集群BF16精度64卡4K序列，MTP1训练吞吐达 **1100 Tokens/p/s**，MFU稳定在 **28.78%**



精简的并行切分策略

采用超节点亲和的**大 EP + 纯 FSDP** 的精简并行切分策略，以极低适配成本和通信开销达成内存占用最优，实现易用性与性能的较好均衡



高效的重型融合算子

针对稀疏注意力和mHC等复杂结构，定制 **Sparse Flash MLA**、**LightningIndexer** 等多个高效的 NPU 融合算子，充分释放芯片稀疏算力



torch.compile + inductor + AutoFuse

依托Ascend C AutoFuse 特性实现端到端的 Vector 算子自动融合，为整网带来 **30%+** 的开箱即用性能收益

Ascend C 融合算子: Sparse Flash MLA

根据输入cmp_ratio不同支持3种Attention计算，分别为SWA、HCA以及CSA。其中Attention部分为Multi-Query Attention:

$$S = Q @ \tilde{K}^T$$
$$m = \max(sinks, \max(S))$$
$$Attention = \frac{e^{S-m} @ \tilde{V}}{\sum e^{S-m} + e^{sinks-m}}$$

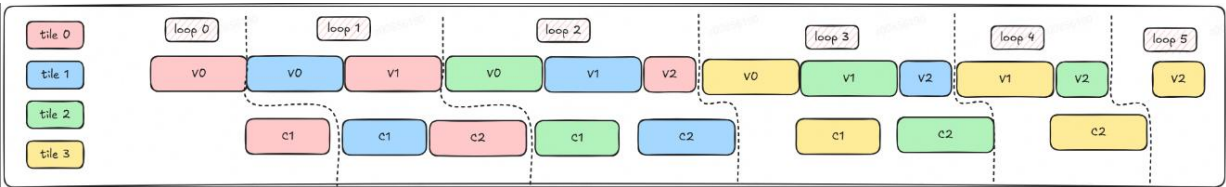
V0 : Gather(cmpkv, topkIndices[i]), 0 <= i < selectBlockCount

C1 : qk = Q @ K^T

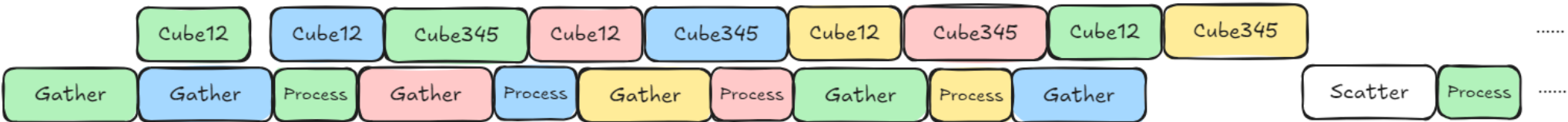
V1 : P = onlineSoftmax(qk, sinks)

C2 : O' = P @ V

V2 : O = rescale(O')



Sparse Flash MLA 的反向算子，主流程可以分为5个阶段，依次为Gather、Cube12、Process、Cube345、Scatter，如图中所示。算子流水排布实现如下，通过Preload一次Gather+Cube12达成CV流水并行的效果。



Ascend C 融合算子: SparseLightningIndexerGradKLLoss

由于 LI 模块进行 Loss 计算时存在巨大内存开销。SparseLightningIndexerGradKLLoss 算子将 Main Attention score 计算、LI 的反向，以及 Loss 计算过程融合，减少中间内存占用，优化内存和性能。

$$I_{t,:} = W_{t,:} @ \text{ReLU}(q_{t,:} @ (K_{t,:})^T)$$
$$\mathcal{L}(I) = \sum_t D_{\text{KL}}(p_{t,:} \parallel \text{Softmax}(I_{t,:}))$$

计算过程可以拆分成 5 个阶段：

1. V0: 依据 Top-k 的索引从 Main Attention 的 K 和 LI 的 K 中提取有效数据
2. C1: 完成 Main Attention 原始 Q 和 K 以及 LI 的 Q 和 K 矩阵运算
3. V1: 完成 KLLoss、dW 计算
4. C2: 完成 LI 反向的 dQ 计算
5. V2: 通过 ScatterAdd 完成 LI 反向的 dK 计算

在实现过程中，针对 V0 进行两次 Preload，并通过 PingPong 掩盖 C1 和 C2 的计算，提升流水的并行度。



DeepSeek-V4 昇腾高性能续训练优化 Highlights



开箱即享的高性能续训练解决方案

CANN 已基于 TorchTitan 支持 DeepSeek-V4 的模型续训练，采用A3集群BF16精度64卡4K序列，MTP1训练吞吐达 **1100 Tokens/p/s**，MFU稳定在 **28.78%**



精简的并行切分策略

采用超节点亲和的**大 EP + 纯 FSDP** 的精简并行切分策略，以极低适配成本和通信开销达成内存占用最优，实现易用性与性能的较好均衡



高效的重型融合算子

针对稀疏注意力和mHC等复杂结构，定制 **Sparse Flash MLA**、**LightningIndexer** 等多个高效的 NPU 融合算子，充分释放芯片稀疏算力

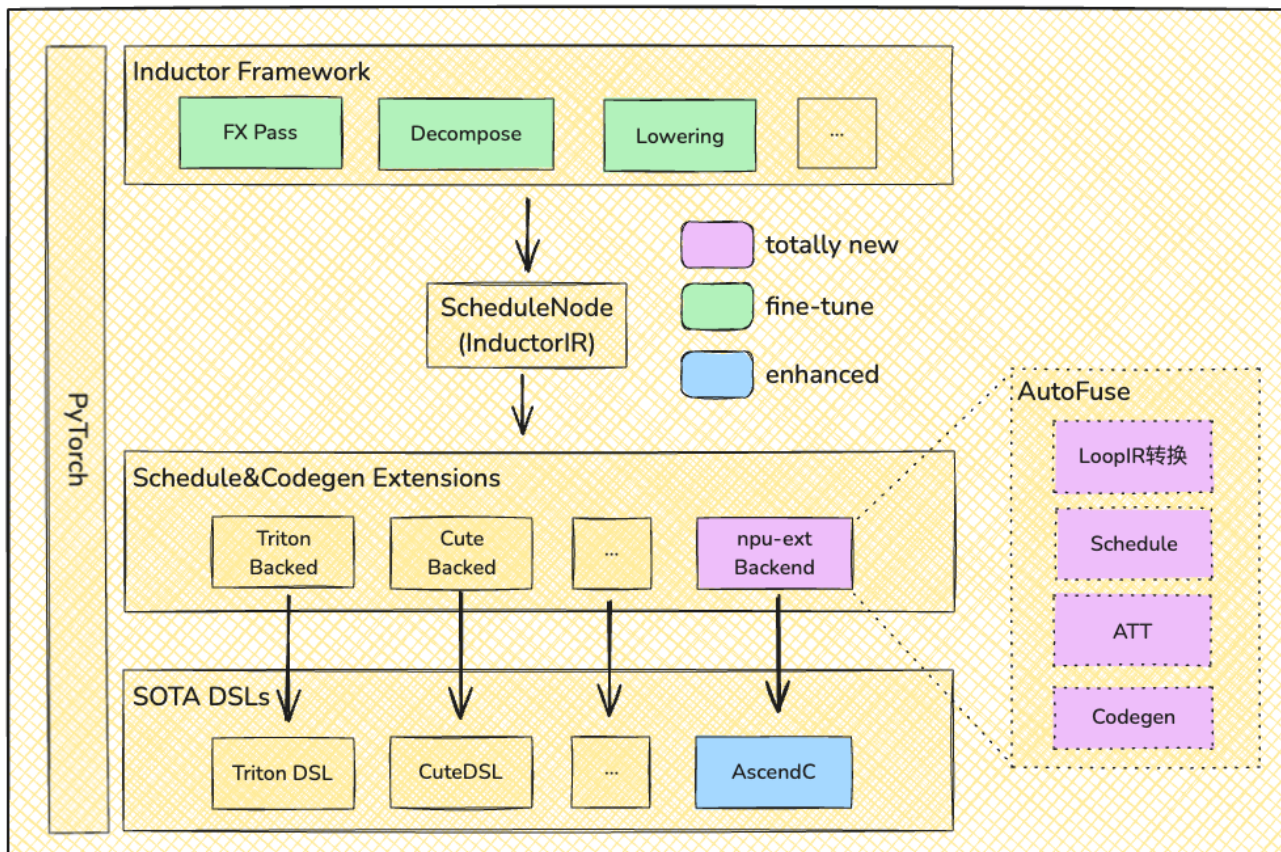


torch.compile + inductor + AutoFuse

依托Ascend C AutoFuse 特性实现端到端的 Vector 算子自动融合，为整网带来 **30%+** 的开箱即用性能收益

Torch.compile + Inductor接入方案

从 FX 图捕获到 AutoFuse 自动生成 Ascend C Kernel



一行使能

PyTorch 前端增加一行代码即可开启入图优化。

NPU 亲和的 Codegen

自动生成硬件架构感知的 Ascend C Kernel, 告别手写。

性能收益

Vector kernel 数量减少 30%, host-bound 消减, H2D/D2H 开销显著降低, 整网性能加速比**1.3x**。

AutoFuse: 基于 Ascend C 的算子 Kernel Codegen 后端



硬件模型：显示管理数据搬运与计算流水

数据载入 (Load)

将数据从全局内存(GM)搬运到片上缓存(UB)。

数据计算 (Compute)

在计算单元上对UB中的数据进行处理。

数据载出 (Store)

将计算结果从UB写回GM。



软件优化：编译器精确计算与协同优化

每次 Load 与 Store 的数据规模 (Tile)

核数和核内迭代空间规模



核心目标：通过数据搬运 (DMA) 与计算 (Compute) 的流水并行重叠，最大化计算单元利用率

- 支持 Inductor 接入：保持生态兼容
- 架构感知优化：亲和 NPU 分级存储和并行机制
- 支持动态 Shape：支持动态 Shape 场景的高性能融合算子 Kernel 代码生成

AutoFuse: 架构设计与流程介绍



多语言 API 交互层

提供 Python 和 C++ 双 API 接口，支持前端计算图的灵活集成与高效调用。



智能调度与优化 (Schedule)

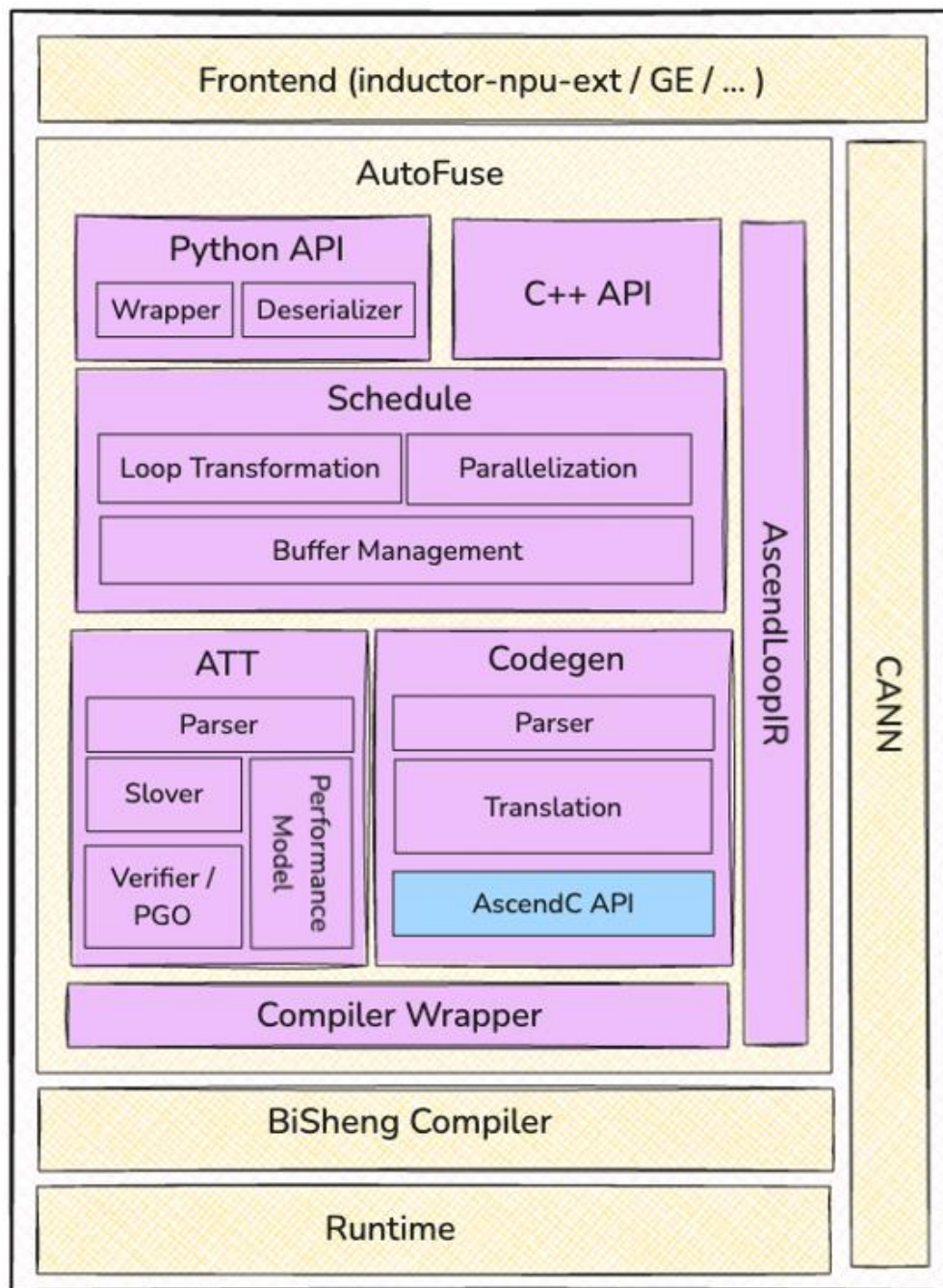
核心优化引擎，通过循环变换、并行化及 Buffer 管理策略，对计算图进行深度架构感知优化。



ATT 寻优与代码生成 (Codegen)

面向硬件感知的自动Tiling参数寻优，Codegen 负责解析与翻译，生成高效的 Ascend C Kernel 代码。

<https://gitcode.com/cann>



CANN

AutoFuse: Schedule——实现NPU吞吐最大化的关键



循环变换

(Loop Transformation)

通过轴分组、分组合并、循环合并、切分、调序等技术，优化计算的**执行结构**与**访存模式**，提升**数据局部性**。



并行化

(Parallelization)

通过循环合并切分和向量化技术，把计算任务映射到多核和 SIMD 指令，提升 NPU **多核执行效率与吞吐能力**。



缓存管理

(Buffer Management)

通过 Bufferization、Double Buffer、缓存与复用等技术，减少 GM 访问并实现**计算-访存流水并行**。



核心目标

在不改变计算语义的前提下，将 Low-level IR 表达的计算图转换为多组架构感知优化的模板计算图。

AutoFuse: Codegen——自动生成高性能Kernel



翻译与映射 Translation

把上层的 AscendLoopIR “翻译”成底层 NPU 硬件指令（Ascend C API），确保语义无损。



指令生成 Generation

依据 Tiling 参数，自动生成循环控制、DMA 数据搬运等指令序列。



跨架构支持 Cross-architecture Support

统一抽象芯片架构，支持不同代际架构的NPU芯片，上层无感知。



性能优化 Optimization

执行指令选择等底层优化，挖掘硬件极致性能。针对Ascend 950新增的 NDDMA、VF 等特性，自动选择高性能指令。

AutoFuse: ATT 极速寻优——启发式分阶段搜索算法

🏆 启发式分阶段搜索

阶段1: UB受限优化

主约束UB容量

阶段2: Core扩展近似

先确定单核结构，多核近似扩展
先优化Tile，再优化Core

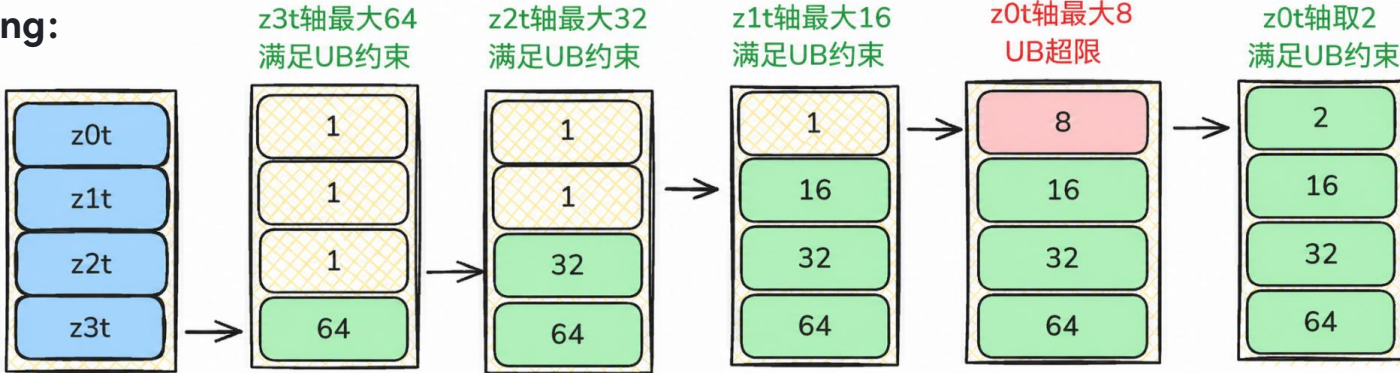
阶段3: 档位离散搜索

性能函数为分段结构
离散空间搜索

🔊 轴排序求解器 (UB优先)

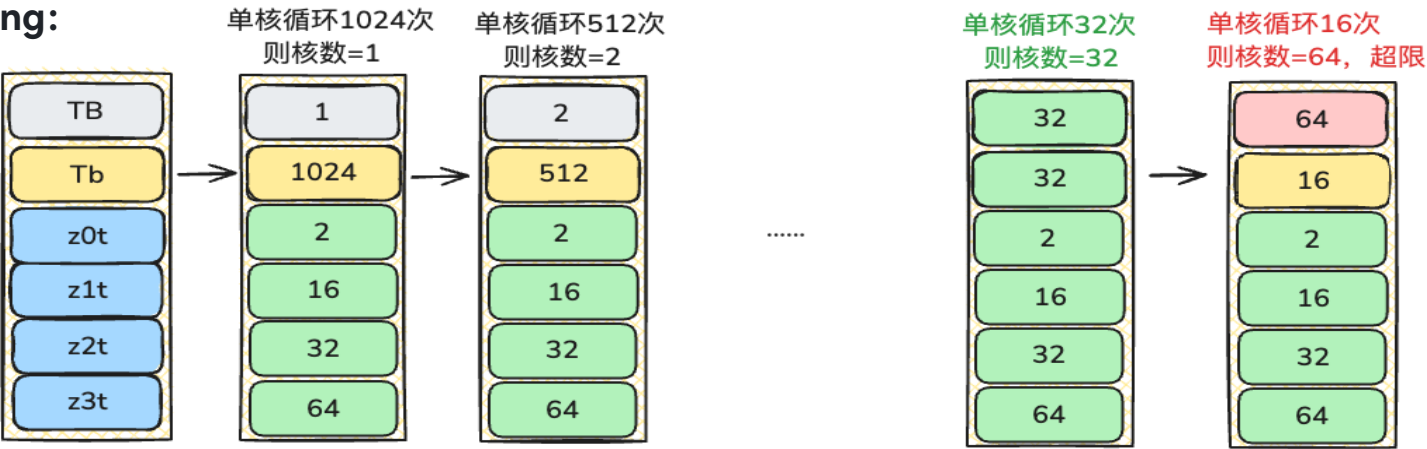
核内Tiling:

目标: UB利用率优先



分核Tiling:

目标: 核数利用率优先



AutoFuse: ATT 全局调优——PGO



核心思想：实测代替理论推导

不依赖完美的理论模型，而是通过在真实硬件环境中对不同Tiling参数组合进行实际性能测量，指导搜索过程。

工作流程：一次迭代，提高效率



01. 采样生成

基于启发式规则，为所有模板生成UB利用率、核数利用率等参数组合。



02. 建模搜索

基于实测数据构建经验模型，评估每组参数组合，筛选TopN。



03. 评估标准

UB与核数利用率平衡
核内循环与核数平衡



04. 真实测量

静态编译 TopN 候选解，
NPU硬件实测，选出最优解。



策略优势

结果准确，自动适应硬件差异，鲁棒性强。



潜在代价

引入一定的编译时间开销，需实际运行程序测量。

AutoFuse: 使能方式

```
1 import torch
2 import torch_npu
3 import inductor_npu_ext
4
5 @torch.compile
6 def test_add_sum(x, y):
7     return torch.add(x, y).sum()
8
9 x = torch.randn(32, 1024).npu()
10 y = torch.randn(1, 1024).npu()
11
12 out = test_add_sum(x, y)
13 golden = (x + y).sum()
14
15 max_diff = torch.max(torch.abs(out - golden))
16 assert max_diff < 1e-3, max_diff
17 print("Succeed")
```

<https://gitcode.com/cann>



依赖组件

CANN 9.0.0.beta1社区版本
torch_npu == 2.10.0rc2
inductor_npu_ext



安装方式

// 极简安装

git clone

<https://gitcode.com/Ascend/torchair.git>

cd torchair/experimental/_inductor_npu_ext/

pip3 install -e ./python/

CANN

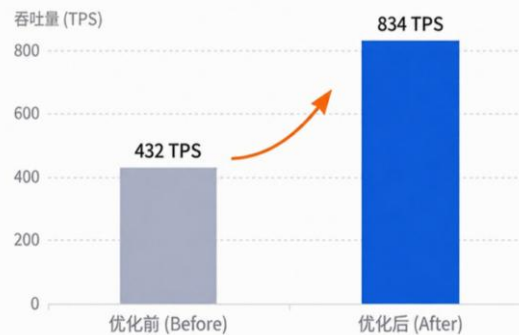
DeepSeek-V4续训练：整网性能收益分析

依托CANN平台+ torchtitan-npu 插件，在A3 64卡集群上完成了DeepSeek-V4的续训练优化。通过引入**纯FSDP+大EP切分**、使能**AutoFuse**，并融合**Sparse Flash MLA**、**LI Loss**等Ascend C算子，成功实现了整网性能的全面突破，充分释放了昇腾硬件的计算潜力。

1. LI等高效NPU融合算子

整网吞吐提升效果：

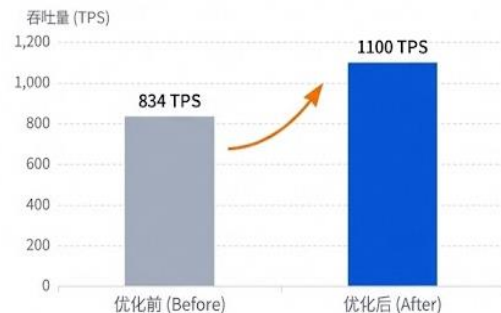
↑ **90%**
Speed up



2. 算子 AutoFuse 特性

整网吞吐提升效果：

↑ **30%**
Speed up



整网吞吐

1100 Tokens/p/s

训练优化的核心产出指标



MFU利用率

28.78 %

计算资源的高效利用



资源空闲占比

2.79 %

极低空闲，算力压榨充分



集群通信开销

12.53 %

A3集群的超高带宽

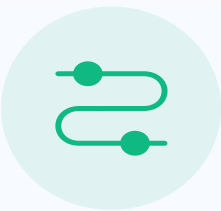
DeepSeek-V4续训练: AutoFuse Speedup 30%



消除“访存墙”

Memory Bound Optimization

将多个算子重构为融合内核，让中间张量驻留于片上缓存(SRAM/L1)，有效规避频繁读写全局显存的带宽开销。



选择低耗时指令

Instruction Selection

通过指令选择技术，将计算效率低的指令组合替换为效率高的简单指令组合。



精简内核启动开销

Launch Overhead Reduction

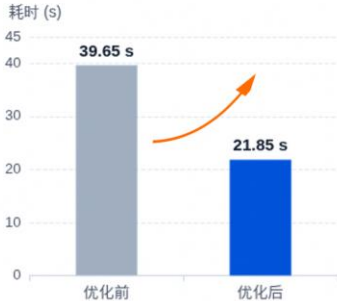
大幅减少Host-to-Device握手交互次数，清理算子间的“计算碎屑”，确保NPU核心持续处于高负载的高效产出状态。

mHC结构AutoFuse拥有更好表现

block	mhc小算子	mhc triton		mhc AF融合	
	耗时(us)	耗时(us)	加速比	耗时(us)	加速比
hc_pre	3814.4	2262.9	1.69	2474.4	1.54
attn	10291.5	9800.8	1.05	9587.6	1.07
hc_post	3303.3	3055.9	1.08	1100.3	3.00
hc_pre	3078.1	1083.5	2.84	1945.6	1.58
moe	26374.2	25009.6	1.05	23095.1	1.14
hc_post	3897.6	2635.8	1.48	1061.3	3.67
total	50759.1	43848.6	1.16	39264.3	1.29

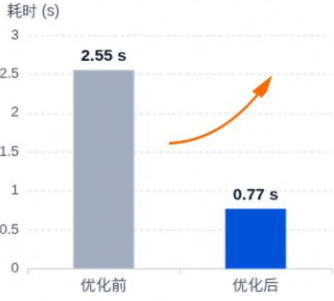
1. Vector 算子耗时

提升倍数
1.81x Speedup



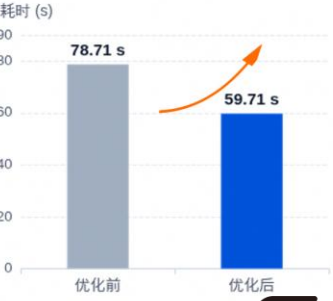
2. Free time 耗时

提升倍数
3.31x Speedup



3. 整网执行耗时

提升倍数
1.32x Speedup



AutoFuse 特性整网收益分析 (耗时对比)

DeepSeek-V4续训练： AutoFuse 核心收益拆解

融合后算子名称	数量 (Count)	融合后 总耗时 (s)	融合前 总耗时(s)	净收益 时间 (s)	优化核心
autofused_mul_sum 系列	10232	1.6	7.1	5.5	带宽优化：消除 Mul 结果写回内存再读回的操作，直接在片上完成规约。
基础逐元素融合 (Mul/Add/Div)	~20 万个	7.8	12.8	5	访存收敛：将大量独立的乘、加、除操作合并，显著减少了对内存的总访问次数。
autofused_add_div_expand_mul_pow	1584	1	2.6	1.6	指令加速：将极高延迟的独立 Pow 算子指令化，通过计算掩盖访存延迟。
BroadcastTo 隐式化 (聚合项)	~3.3 万个	0.3	2.5	2.2	隐式广播：通过 Stride 逻辑变换实现广播，彻底消除了 2s 多的物理内存拷贝耗时。
autofused_npu_dtype_cast 系列	14832	1.8	2.5	0.7	调度简化：吸收数万次类型转换操作，大幅降低了 CPU 下发任务给 NPU 的频率。

DeepSeek-V4续训练: AutoFuse片段解析 (1)

收益分析

融合范围

水平融合数: 4 路

融合算子数: 17个

算子类型: Elementwise、Broadcast、Cast

融合收益

基线耗时: 6138.8 us

开AF耗时: 587.7 us

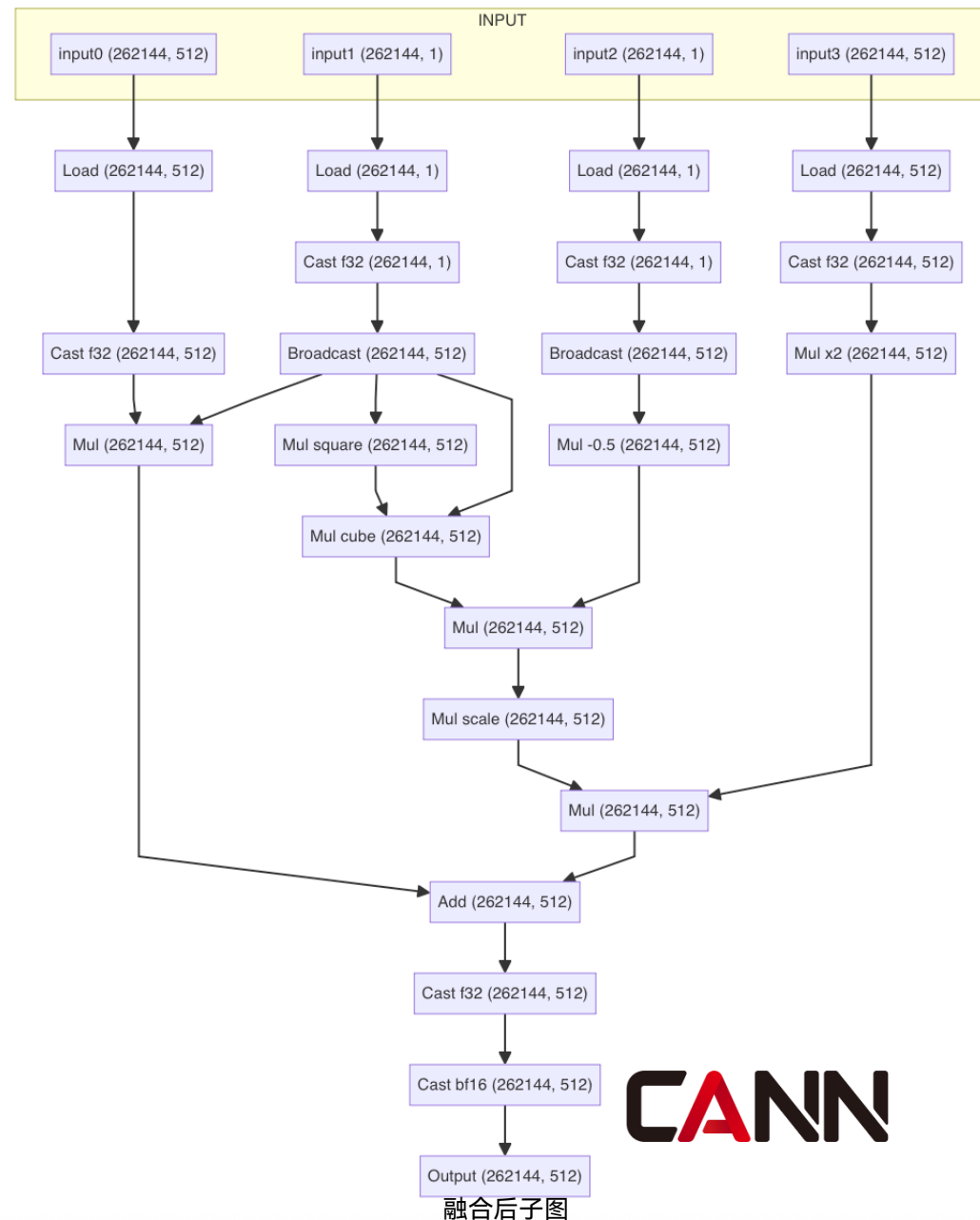
加速比 : 10.44 x

收益分析

循环合轴: 原 Shape(1, 4096, 64, 512) → 合并为(262144, 512)

等式变换: 原图两次 Pow → 改成两次 Mul, Mul 算子效率更高

多路融合: 原4路输入 → 全部融合, 中间结果不出 UB



DeepSeek-V4续训练: AutoFuse片段解析 (2)

收益分析

融合范围

水平融合数: 2 路

融合算子数: 6 个

算子类型: Elementwise、Cast、ReduceSum

融合收益

基线耗时: 1033.5 us

开AF耗时: 499.9 us

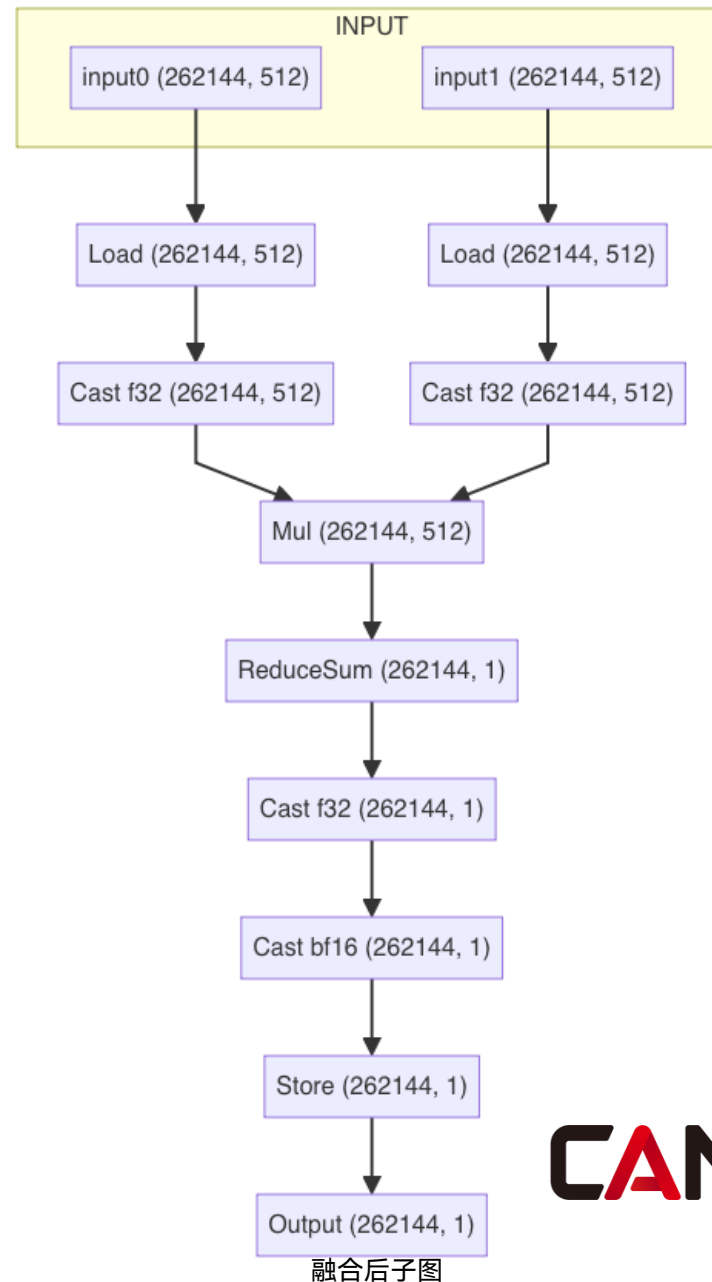
加速比 : 2.07 x

收益分析

循环合轴: 原 Shape(1, 4096, 64, 512) → 合并为(262144, 512)

Reduction融合: 尾轴 Reduction, UB 支持全载 R 轴, 数据不出 UB

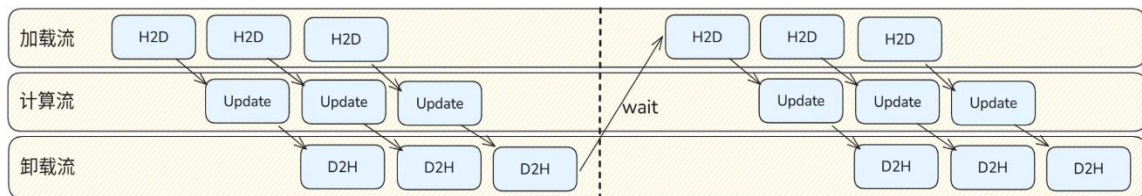
<https://gitcode.com/cann>



CANN

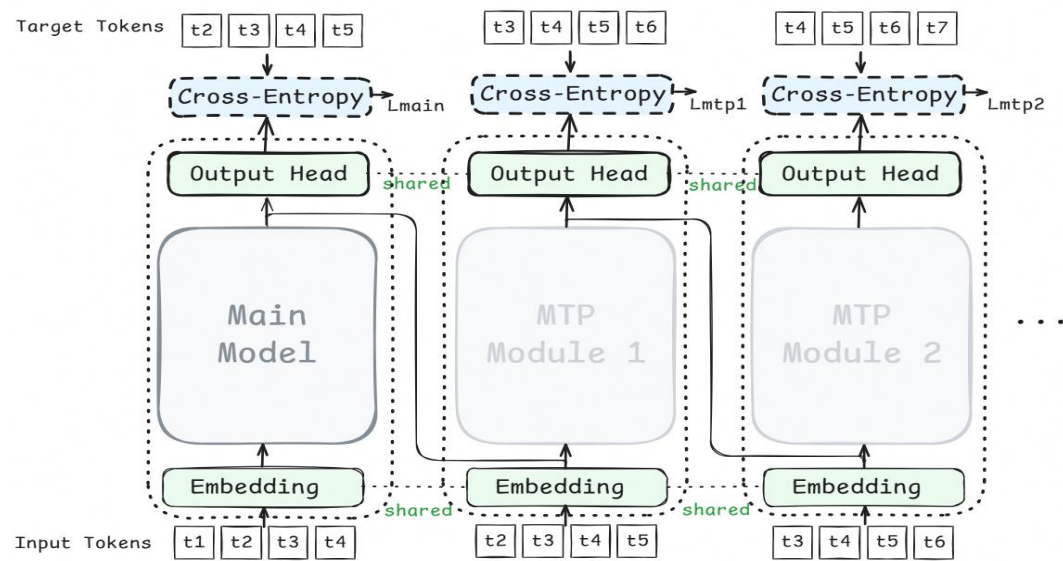
DeepSeek-V4续训练复现：基础特性实现

Swap Optimizer(优化器显存卸载)



- 为节省device内存，在前反向期间将优化器卸载至host内存。
- 以param为粒度将“优化器加载——优化器更新——优化器卸载”设计为切片流水并行的形式。

MTP(Multi-Token Prediction)



- 基于 torchtitan 训练框架与 DeepSeek-V4 模型结构，实现了 MTP 模块的定制化适配。
- 无缝兼容模型原有的 FSDP、EP、TP 等分布式并行策略与训练逻辑。

DeepSeek-V4续训练复现：Step by step 指导

✓ 跟随README文件，丝滑复现DeepSeek-V4 Flash续训练



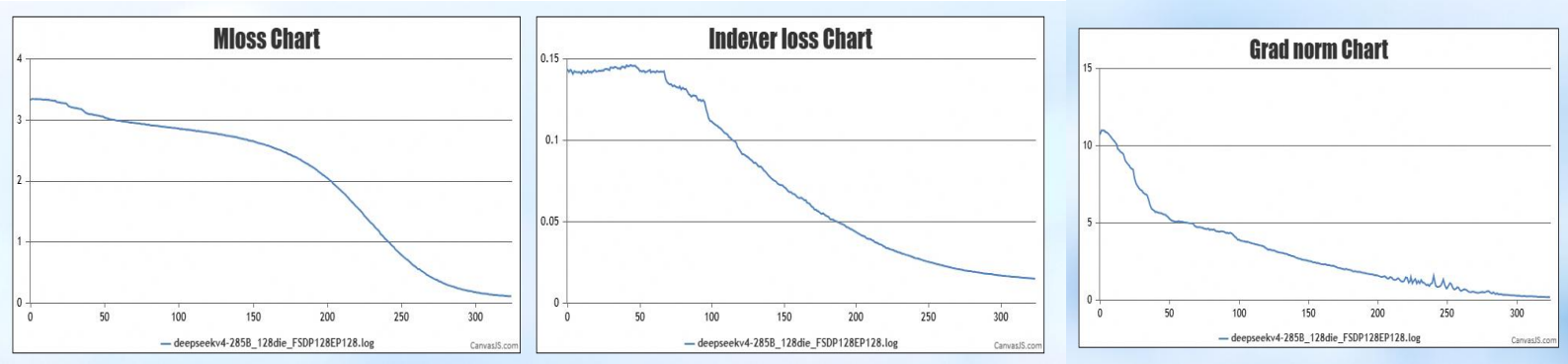
DeepSeek-V4续训练README



从[镜像地址](#)中下载 docker 镜像【dsv4_train_torchtitan:cann9.0.0.beta.1_v1.0】
据README指导，安装如下环境

torchtitan-npu版本	torchtitan版本	PyTorch版本	torch_npu版本	CANN版本	Python版本	Triton Ascend
master (主线)	0.2.2	2.10.0	2.10.0rc2	9.0.0-beta.1	Python3.11.x	3.2.0

续训练效果参考如下



Future Plan

已开启

[Roadmap] torchtitan-npu Q2 2026 roadmap

#5

hanboyou

创建于 16 天前

hanboyou

成员

16 天前 创建

26年Q2概览

本季度 torchtitan-npu 聚焦增强基础编译能力与性能优化；同时探索 torchtitan 作为RL训练后端的可能。

特性与能力

状态	特性	支持状态
New	torchtitan-npu 正式开源	已支持
New	Inductor+Autofuse 自动融合能力	已支持
New	DeepSeek-V4-Flash模型支持	已支持
SOON	DeepSeek-V4-Pro模型支持	未支持
SOON	Muon优化器训练支持	未支持
SOON	支持FP8和A8W4的低精度训练(Ascend 950)	未支持
SOON	接入 veRL 社区，使能 RL 训练阶段 torchtitan 后端	未支持
SOON	整网性能开箱 1.0x Megatron	未支持

26年Q3概览

状态	特性	支持状态
SOON	DeepSeek-V4 训练支持 Mega MoE	未支持

已开启

AutoFuse Roadmap(2026 H1)

#44

wangxiaotian995

创建于 8 分钟前

wangxiaotian995

成员

8 分钟前 创建

历史修改

AutoFuse Roadmap(2026 H1)

AutoFuse基础融合能力已基本完成构建。H1计划围绕A5形态融合能力增强与融合效果泛化加固的角度展开工作。以下描述均默认表述在A5形态：

1.泛化加固

状态	描述	关联issue
	计算类算子（elemwise、broadcast、reduce）融合能力完成泛化验证	TODO
	A5默认开启自动融合	TODO

2. 支持mm+elemwise融合

状态	描述	关联issue
	mamtul系列算子与elemwise算子融合完成基础能力构建	TODO

3. Vector融合能力增强

状态	描述	关联issue
	norm结构融合能力增强，支持多reduce多broadcast融合结构	TODO
	支持concat类与计算类算子的融合能力	TODO
	支持split、slice类与计算类算子的融合能力	TODO

4. 易用增强

状态	描述	关联issue
	后端处理pass化，上提AscendLoopIR显示表达	TODO
	过程透明化，过程处理统一格式dump输出，确保dump观察点无断点	TODO

<https://gitcode.com/cann/torchtitan-npu/issues/5>
<https://gitcode.com/cann/graph-autofusion/issues/44>

欢迎体验和贡献

torchtitan-npu

基于 torchtitan 的昇腾全流程大模型训练适配插件

docs latest license BSD 3-Clause CONTRIBUTING SIG framework-adapter pypi 0.2.2.post1

简介

torchtitan-npu 定位为 torchtitan 的昇腾（Ascend）后端扩展插件，通过即插即用的硬件亲和性优化，充分释放NPU算力，助力 PyTorch native 训练在昇腾平台无缝、高效、稳定地运行。

本插件基于社区 ModelConverter 拓展机制构建，已支持多维度训练优化，涵盖 NPU融合算子、图优化、图下沉、算子自动融合、显存管理、分布式并行以及调试维测能力等等。

社群

SIG framework-adapter

SIG 例会: [sig-framework-adapter](#)



torchtitan-npu社区微信交流群



deepseek v4 续训练昇腾适配技术报告



torchtitan-npu开源仓地址



Graph-AutoFusion开源仓地址

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.